15-09-25

* Relational Data model

- The relational model is a model that represent How data is stored in a relational database.

- A relational database stores data in the form of tables. (Relations).

- A table is a collection of Columns and rows. Where Column represent attributes of an entity and row represent records.

- The relational model proposed by E.F Codd in 1959

- The relational model is most widely used data model in the world because it is very simple so programmer can work easily in it. and this model fulfills all the needs of data processing and storage.

Tuple ←

ID	Name	age	Phone No.
1	Amit	22	1234567890
2	Ankush	18	2345678901
3	Ankur	28	3456789012
4	Sneha	21	4567890123
5	Yogesh	20	

Date:
Page:

* Some important topics of relational model

1- Attribute - Attribute is a property that define a relation (table)

Ex- In the above table; Student - Id, Name, age, phone No are attributes of the table.

2- Tuple - Each row of the table is called a tuple. there are five tuples in the above table, One tuple is ~~stored~~ shown below.

3- Degree - The number of Attributes in a tuple is called the degree of table.

The degree of the table is Student is 4

4- Cardinality - The number of tuples in a table is called Cardinality of the table

The Cardinality of the table student given above is 5.

5- Column - The Column represent the value of a particular attribute

Here Column of the table student is shown 1, 2, 3, 4, 5.

6- Null Values → The value which is not available is called Null Value.

Ex- You can see in the table Student - the phone_No Value of ID 5 is NULL.

16-09-25

7- Relation Schema - The relation schema represent the attributes of the table along with its name
Ex- Student (ID, Name, age, phone No) - is a relational schema for the table Student.

* Relational Model Constraints.
Every relational has some Condition which we call relational integrity Constraints.

1- Key Constraints.
2- Domain Constraints.
3- Referential integrity constraints.

* Integrity Constraints.
When we store data in a relational Database we have to ensure that the data is correct, accurate and logical.

To ensure this process, we use integrity Constraints.

These Constrainst are some rules that maintain the Validity and Consistency of the data entered into a table

There are some dicsipline rule for every class in school, so that every thing remain Correct & under Control.

Similarly in the database ~~to~~ these ~~two~~, ~~each~~ Constraints maintain the correctness and reliability of data.

* Type of relational integrity Constraints.
- Domain Constraint - This Constraints ensure that the data, entered in an attribute is according to its define domain.

ex-1- if we have kept the type of column as integer then only numbers should come in it.

ex-2- Age should be integer, so we can't enter text or date in it.

- Entity integrity Constraints - This Constraint is related to the primary key
No value of the primary key in any table should be null because it identifies the row if roll no is the primary key in the student table then roll no can never be empty (null)

- Referential integrity Constraints - This Constraints Controls the relationship between two tables
Whenever a foreign key of one table references

18-09-25

the primary key of another table then this constraints check that the value in the foreign key should be present in other table

Ex- If roll-no in the addition table is the foreign key that points to the primary key of the student table they any roll no can be added in the addition table only if it already exist in the student table.

→ Key Constraints - Key Constraints insure that the primary key or Candidate key in a table should not have duplicate value.
Each row should have a unique identity
ex - If email is the candidate key then the email address of two students cannot be the same.

→ Not null Constraints - This Constraints restricts any field from null value.
This means it is necessary to enter some value in that Column

ex - If the name Column is not null then we can't leave the name of any student blank

→ Unique Constraints- This Constraints ensures that every value entered in a column should be unique.
In this null is allow but only ones.

ex- If phone no is unique then the number of every student should be different.

* Relational Algebra :-
- Relational algebra is a set of operations that are used to manipulate data using Relation.
- Relational algebra is a procedural language.
- In this operator are used to perform query and the intermediate language used in DBMS.
- Relational algebra have the following operator

(i)- Union (∪)
(ii)- Difference (−)
(iii)- Selection (σ)
(iv)- Projection (π)
(v)- Cartesian Product (x)
(vi)- Intersection (∩)
(vii)- Rename (ρ)
(viii)- join

- Union (∪) - The Union operator is represented by '∪' symbol.

It's Syntax is as follows:-

$R = P \cup Q$

Where P and Q are two input relations and R is the output relation.

Union operation has two input relation which are Union Compatiable. The output relation of this operation Contains those tuples (roles) which are in relation 1 and relation 2 or in Both and the tuples which are duplicate are eliminated

R →

A	1
B	2
D	3
F	4
E	5

S →

A	1
C	2
D	3
E	4

R ∪ S →

A	1
B	2
C	2
D	3
E	5
F	4
E	4

- Difference (-) - Difference operator is represented by '-' Symbol. It's syntax is as follows.

  $R = P - Q$.

Difference operation removes the common tuples present in the relation 1.
The output that we get will be the tuples present in relation 1 which are not present in relation 2.

We can easily understand this through the following figure.

R →

A	1
B	2
D	3
F	4
E	5

S →

A	1
C	2
O	3
E	4

R - S →

B	2
C	2
D	3
F	4
E	5

S - R ⇒

C	2
O	3
E	4

- Selection (σ) → It is represent by 'σ' Symbol. This operator is used to select the tuple which satisfy given the condition.

This operation is unary operation which is define in only one relation or table

→ σ topic = 'Database' (student)
O/p → selects tuples from student where topic = 'Database'

→ σ topic = "Database" and author = "CSESTUDIES" (student)
O/p → selects tuples from ~~where student~~ Student ~~student~~ where author = "CSESTUDIES"

~~σ topic~~ σ sales > 5000 (Customers)
O/p → selects tuples from Customers where sales is greater than 5000;

- Projection (π) → It is represented by 'π' symbol.
It is a unary operation that is it has only one relation.
The projection operator select us the subset of attribute from a relation and the attribute which are not selected are eliminated.

Customer_ID
1
2
3

Customer-Name	Status
Suraj	Active
Rahul	Active
Amardeep	Active

Customer_Name Status (Customer)

- Cartesian Product (X) → It is represented by 'X' symbol

It's dyntax is as follows.

$R = P \times Q$

This operation is used to combine the ~~operation~~ information of two different relations into one relation.

R →

A	1
B	2
D	3
F	4
E	5

S →

A	1
C	2
D	3
E	4

R X S →

A	1	A	1
A	1	C	2
A	1	D	3
A	1	E	4
B	2	A	1
B	2	C	2
B	2	D	3
B	2	E	4
D	3	A	1
D	3	C	2
D	3	D	3
D	3	E	4

F	4	A	1
F	4	C	2
F	4	D	3
F	4	E	4
E	5	A	1
E	5	C	2
E	5	D	3
E	5	E	4

- Intersection ($\cap$) → It is represented by '$\cap$' Symbol This operation is use to Select Common tuples from two relations
It's Syntax is as follows.

$R = P \cap Q$

This operator is used to Combine the information of two different relations into one relation.

P →

A	1
B	2
D	3
F	4
E	5

Q →

A	1
C	2
D	3
E	4

$P \cap Q =$

A	1
D	3

- Rename (P) → It is represented by 'P' Symbol This operator is used to rename the relation.
- Join - This operator Combines two relations into a new relation. The join operators are of the following types-

1- Theta Join
2- Right outer join
3- Left outer join
4- Full outer

19-09-25

5- Natural join

* Relational Calculus → In a database management System, relational calculus is a non procedural query language that does not require us to specify how to retrieve data, but rather what data we need.

→ Relational Calculus allows us to retrieve and insert data for a table.
→ It is based on the relational model and usage logic to search and retrieve data from a database.

example - SELECT the tuples from STUDENT relation with CLASS = "COMPUTER"

In the above example you are given a relational calculus that select tuples from the student relation containing class = "COMPUTER"

* Types of Relational Calculus.
1- Tuple Relational Calculus
2- Domain Relational calculus

1- Tuple Relational Calculus - TRC is a Non-procedural query language used to select tuples (Rows) from a

table ~~with~~ based on a specified Condit-
ions.

→ Variables are used to represent tuples
→ Below the Syntax for tuple relational Calculus ~~(TRC)~~ (TRC)

Syntax-

$$\{t \mid P(t)\} \text{ or } \{t \mid \text{condition } (t)\}$$

In the above Syntax (t) represent a tuple and P(t) represents a condition.

2- Domain Relational Calculion - DRC is used to select Values from the table based on certain Conditions.

→ Variables are used to represent values.
→ Below is the Syntax for Domain relational Calculus

Syntax -

$$\{\langle P(x_1, x_2, x_3 \ldots x_n)\}$$

In the above Syntax $x_1, x_2, x_3, \ldots x_n$ are called Domain Variables and are used to select values.

$P(x_1, x_2, x_3 \ldots x_n)$ is called the Predicate condition.

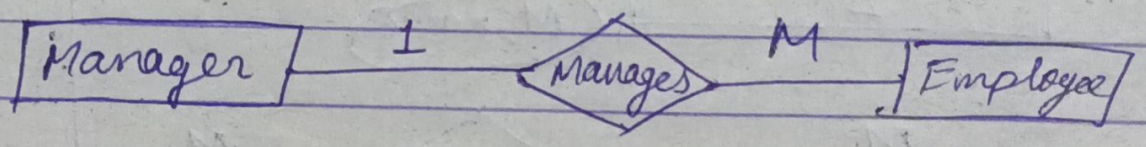
Date:
Page:

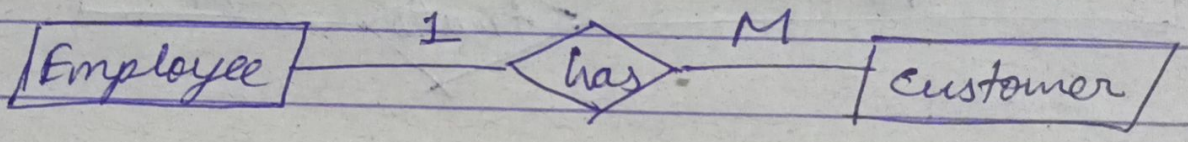
* Draw the ER diagram of Bank System

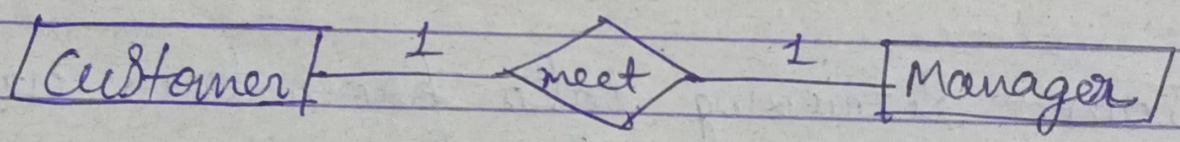
Step-1- Identify the active element.

→ Manager
→ Customer
→ Employee
→ Bank

Step-2- Identify the relation between them







Step-3- Identify the key attribute

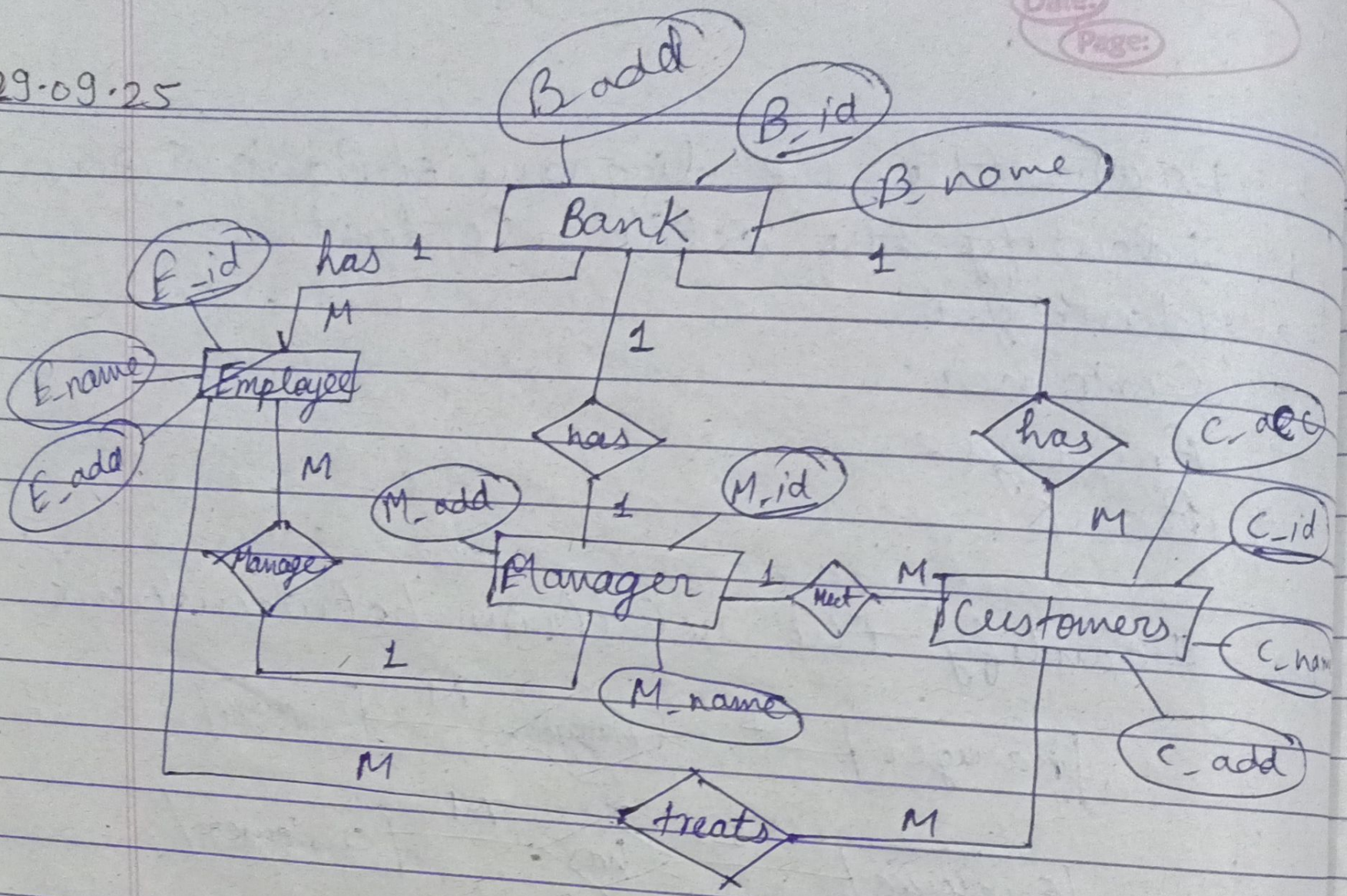
Manager → M_id
Employee → E_id
Customer → C_id
Bank → B_id

Step-4- Identify the other attribute

Manager → M_name, M_age, M_add
Employee → E_name, E_age, E_add
Customer → C_name, C_Acc, C_add
Bank → ~~Bank~~ B_name, B_add

Step-5- Draw the ER diagram

29.09.25



* Join - Retrieving data from two or more table is called a join.
- In other words joining rows from two or more tables based on a column is called a join.
- Joins are used to combine data from two or more tables
- When we need to retrieve related data from one table from another we use sql join
- The main purpose of a join in sql is to retrieve data from multiple tables.

Example- Employee table

E_NO	E_name	Address
1	Ram	Delhi
2	Varun	Chandigarh
3	Ravi	Chandigarh
4	Amit	Delhi
5	Nitin	Noida

Department table

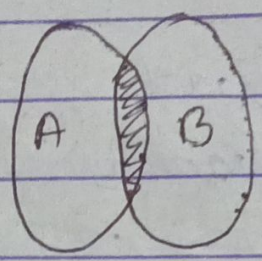
Dep.NO	Name	E_NO.
D_1	HR	1
D_2	IT	2
D_3	Marketing	4
D_4	finance	5

Q- Find employee name of employee whose working in HR Department.

→ Select address from Employee where E_name= 'varun';

* Types of join- There are many types of join
1- Inner Join
2- Left Join
3- Right Join
4- Full Join
5- Natural Join

1- Inner Join - Inner join selects only those rows from both tables whose values are the same.



Syntax - select table1.column 1 , table1.column 2,
table2.column 1, ——
from table 1
INNER JOIN table 2
On table1.matching_column = table2.
matching_column;

Table 1 - Employee Table

Emp_Id	Emp_Name	Dept_id
1	Raj	101
2	Anita	102
3	sohan	103
4	Mira	101

Table 2 - Department table

Dept_Id	Dept_Name
101	IT
102	HR
104	Finance

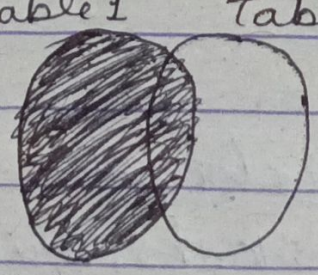
→ Result of Inner Join

Emp_Id	Emp_Name	Dept_Name
1	Raj	IT
2	Anita	HR
4	Mira	IT

```
Select Employee.Emp-Id, Employee.Ename, Employee.
       Dept_name
from Employee
Inner Join Department
On Employee.dept_id = Department.dept_id;
```

Note→ Inner Join only returns rows where the join condition matches in both table

2- Left join - It returns all rows of the left table and matching rows of the right table. It is also called left outer join



Syntax- Select table1.column1, table1.column2,
table2.column1, ____

```
from table1
LEFT JOIN table2.
On table1.matching_column = table2.matching_
                            column;
```

Table 1 - students table

Stu_id	stuname
1	Alice
2	Bob
3	charley

Table 2 → Marks table

Stu_id	Subject	Marks
1	Math	90
2	Math	80

Result of left join

stu_id	stu_name	Marks	subject
1	Alice	90	Maths
2	Bob	80	Maths
3	Charley		

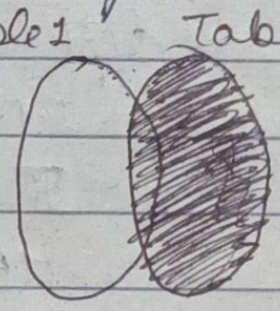
```
Select Students.stu_id, Students.stu_name,
       Marks.stu_id, Marks.subject, Marks.mark
from students
LEFT JOIN Marks.
On students.stu_id = Marks.stu_id;
```

2 Difference between Inner Joins Vs left join

Only matched Rows.

Inner Join	Left join
Only matched rows.	All rows from left table

3- Right Join - It returns all rows of the right table and matching rows of left table. It is also called right outer join



Syntax: Select table1.column1, table1.column2,
table2.column1, ——
from table1
RIGHT JOIN table2
On table1.matching_column = table2.matching_column;

Table 1 → Employees Table

emp_id	emp_name	dept_id
1	Raj	101
2	Anita	102
3	Sohan	103
4	Mira	101

Table2 → Department table

dept_id	dept_name
101	IT
102	HR
104	Finance

30-09-25

Result of right join

emp_id	emp_name	~~dept_id~~	dept_Name
1	Raj	1	IT
4	Mira	4	IT
2	Sohan		HR
4	Mira		

emp_id	emp_name	dept_Name
1	Raj	IT
4	Mira	IT
2	Anita	HR
3		Finance

Select employee.emp_id, employee.name, employee.dept_id, Department.dipt_id, Department.dept_name.
from Employee
RIGHT JOIN Department
on employee.dept_id = Department.dept_id;

4- Full Join - It is made up of left join and right join this returns all the rows from both tables.

Syntax - SELECT table1.column1 , table1.column2,
table2.column1, ----
FROM table 1
FULL JOIN table2
On table1.matching_column = table2.matching_
Coleimn;

Example - Employees Table

emp_id	emp_name	dept_id
1	Raj	101
2	Anita	102
3	Sohan	103
4	Mira	101

Department table

dept_id	dept_name
101	IT
102	HR
104	Finance.

Result of Full Join

emp_id	emp_name	dept_id	dept_name
1	Raj	101	IT
2	Anita	102	HR
3	Sohan	103	
4	Mira	101	IT
5		104	Finance.

Select employee.emp_id, employee.emp_name,
~~employee.dept_id~~, Department.dept_name
from employee
full join department
On employee.dept_id = department.dept_id;

Note - key difference.
Inner Join - Only matched rows.
Left Join - All rows from left + Matches from right.
Right Join - All rows from right + Matches from left
Full Join - All rows from both, unmatched rows get null.

5- Natural Join - This Join joins two tables based on Common Columns.
It returns all rows with matching values in those columns.
Each table has certain business rules for inserting data. These business rules are called data constraints.

Example - Employees Table

emp_id	emp_name	dept_id
1	Anita	10
2	Sohan	102
3	Mira	103
4	Raj	4101

Department table

dept_id	dept_name
101	IT
102	HR
104	Finance

Result of Natural Join

~~emp_id~~	emp_name	~~dept_id~~	dept_name
~~1~~	Raj	~~101~~	IT
~~2~~	Anita	~~102~~	HR
4	Mira	~~104~~	IT

explanation - Natural Join automatically uses department_id as the join column because it ~~exists~~ exists in both tables

Only matching rows are returned (like as inner join)

Sohan (dept_id → 103) and Finance (dept_id → 104) are excluded because there is no match

Note → Natural join can be convinient but dangerous if multiple column have the same name unintensionally.

Most developers prefer explicit inner join ON to avoid mistakes.

Syntax -
```
SELECT Name, Dept_name
FROM employee
NATURAL JOIN Departments;
```

Q- Table 1 → students table

stu_id	name	Course_id
1	Amit	101
2	Neha	102
3	Raj	103
4	Simran	101

Table 2 → Courses table

Course_id	Course_name
101	DBMS
102	OS
104	Computer Network

find the inner join, left join, right join, full join and Natural join with their syntax.

(i)- Result for Inner Join

stu_id	name	Course_name
1	Amit	DBMS
2	Neha	OS
4	Simran	DBMS

```
Select students.stuid, students.name,
                Courses.Course_name
From students
Inner Join Courses
On students.Course_id = Courses.Course_id
```

(ii)- Result of Left Join

stu_id	name	~~Course_id~~	Course_name
1	Amit	~~101~~	DBMS
2	Neha	~~102~~	OS
3	Raj	~~103~~	~~DBMS~~
4	Simran	~~101~~	DBMS

```
SELECT students.stu_id, students.name,
       ~~Co~~ ~~student.Course_id~~, Courses_name
from students
LEFT JOIN Courses
On students.Course_id = Courses.Course_id;
```

(iii)- Result for Right Join

Stu_id	name	Course_name
1	Amit	DBMS
4	Simran	DBMS
2	Neha	OS
~~3~~	~~Raj~~	Computer Network

```
SELECT Students.stu_id, students_name,
       Courses.Course_Name
from students
RIGHT JOIN Courses
On students.Course_id = Courses.Course_id;
```

(iv)- Result for full Join

stu_id	name	Course_name
1	Amit	DBMS
2	Nelva	OS
3	Raj	
4	Simran	DBMS
		Computer Network

```
SELECT students.stu_id, students.name,
        Courses, Cours_name
from students
FULL JOIN Courses
On students.Cours_id = Courses.Cours_id;
```

(v)- Result for Natural Join

name	Course_name
Amit	DBMS
Nelva	OS
~~Raj~~ Simran	DBMS
~~Simran~~	

```
SELECT name, Course_name
FROM student
NATURAL JOIN Courses;
```

6-10-25

* Functional Dependencies.

Functional Dependencies are method which describes the relationship between attributes

dependend ←

$$X \longrightarrow Y$$

determinant ← X, Y → determines

X determines Y

OR

Y determined by X

In other words Y is a subset of X.

Case-1- Valid

Sid	Sname
1	Suraj
2	Suraj

Case-2- Valid

Sid	Sname
1	Suraj
1	suraj

Case-3- Valid

Sid	Sname
1	Suraj
2	chandan

Case-4- Not Valid

Sid	Sname
1	suraj
1	chandan

- Functional dependencies is a very important concept in DBMS When we talk about a relational database, functional dependencies defines how one attribute depends on another attribute.

- Simply put, if we have a relation (table) containing two attributes, X and Y, we can say that Y is functionally dependent on X if there is only

unique y for each x, we write this x as

$$X \rightarrow Y$$

This means we can uniquely identify y from x. This dependence indicates that we can determine the value of Y from the value of X.

* Why functional dependency is important?

→ • It helps her understand the structure of a relation
• Knowing functional dependency is very important in normalization.
• It tells us which attribute are logically related to each other.
• This helps minimize data redundency

* Representation of functional dependency
Functional dependency is represented by an arrow '→' such as

Student-ID → student-Name

It means that a Unique student-name depends on every Student-Id

* Types of functional dependency

(i)- Trivial When an attributes dependends on itself or subset of itself.

$A \rightarrow A$ ✓

or

$\{A, B\} \rightarrow A$ ✓

eg- $A \rightarrow B$

B is subset of A.

Property- LHS ∩ RHS ≠ ϕ

Sid Sname → Sid

(ii)- Non-Trivial functional dependency when the attributes on the right-hand side is not included into the left hand side

eg- $A \rightarrow B$ and $A \neq B$

B is not subset of A.

$A \cap B = \phi$

Sid → Sname

Sid → Ssemester

Sid → Sphone-no.

9-10-25

* Properties of functional dependency
(i)- ~~Reflexivity~~ Reflexivity
(ii)- Augmentation
(iii)- Transitive (M.M. Important)
(iv)- Union
(v)- Decomposition
~~(vi)- pseudo~~
(vii)- ~~Tran~~ pseudo - Transitivity
(viii)- Composition

1- Reflexivity → if Y is subset of X
then ~~xxy~~ X → Y

2- Augmentation - if X → Y then XZ → YZ
Sid → Sname
SidSPhone → Sname Sphone

3- Transitive → if X → Y and Y → Z then
X → Z

4- Union - if X → Y and X → Z then
X → YZ

5- Decomposition - if X → YZ then
X → Y and X → Z

6- ~~Pseudo~~ - (XY → Z . X → Z, Y → Z) X
when we do not decompose in
left hand side

6- Pseudo - Tansitivity → if $X \to Y$ and $WY \to Z$ then $WX \to Z$

7- Composition - if $X \to Y$ and $Z \to W$ then $XZ \to YW$

* Types of functional dependencies:-

(i)- When we talk about dependencies of relation within a database, there are several types of functional dependencies

(ii)- Each type ~~of~~ has its own specific purpose and each dependencies describes how the attributes in a table are interconnected.

(iii)- if you want to understand normalization properly it's crucial to understand all these types thoroughly.

1- Trivial functional dependency. A trial functional dependency is one in which an attribute on the right hand side (RHS) is also part of the left hand side (LHS). We can understand it this way:-

ex- ~~A → A~~ $\{A, B\} \to A$

$A \to A$

explanation- This dependency is always valid because an attribute always depends on itself.

This is mostly a theoritical concept but it's important to understand because it can be used to prove to other dependencies.

2- Non-Trivial functional Dependencies.
Non-Trivial functional dependencies occurs when the RHS is not part of the LHS i.e the attribute it depends on is completely different.

ex- $A \rightarrow B$ (where B is not part of A)
$\{A,B\} \rightarrow C$ (where C is not part of either A or B)

This type of dependency helps to defines normalization rules and is useful for controlling data redundancy

3- Full-functional dependency
When a non-prime attribute completely depends on entire candidate key. and not all any part of the key, it is called a full functional dependency

Sid - student_id SubCode → subject code

ex- {Sid, SubCode} → Marks

explanation Here Marks doesnot depend on either student_id or subject_code, but on both simultaneously.

This dependency is very important in 2NF because the objective of the 2NF is to remove partially dependencies and achieve full dependencies.

4- Partial functional dependencies:
When an non-prime attribute depends only on a part of the Candidate key, it is called a partial dependency.

This dependency tells us that there may still be redundancy in the relation.

ex- {Roll_NO, Subject} → Marks.

Roll NO → Student Name

explanation- Here student name depends only on Roll_NO Not on the entire key ({Roll_NO, Subject})

This dependency distinguises 1NF from 2NF

5- Transitive functional dependency:
Transitive dependency occurs when one attributes depend on another and that second attribute then depend on a third, this dependency is mostly removed in 3NF.

ex- $A \rightarrow B$, $B \rightarrow C$, $A \rightarrow C$ (transitive dependency)

explanation- if C is non-prime attribute and depends indirectly on A through B, this is called a transitive dependency.
This is necessary to remove this attribute. to prevent redundancy and anomalies in the database.

6- Multivalued dependency
It occurs when an attribute has multiple independent values that are not related to other attributes This dependency is used in 4NF.

ex- student Id → Hobby

explanation- This means that a student ID can have more than on Hobby and these hobbies do not depend on any other attribute (such as address)

10-10-25

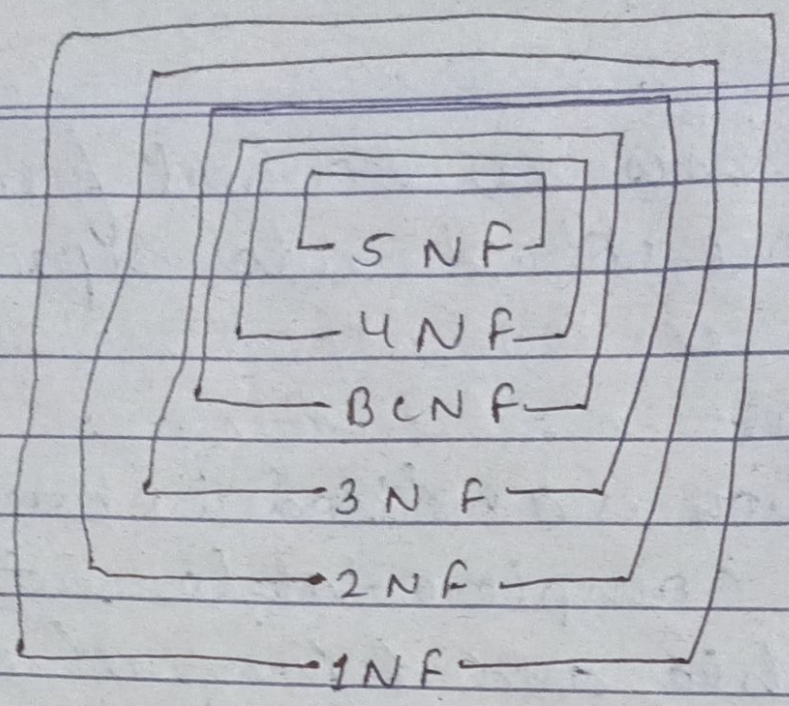
To Convert this to normal form, we need to break it into separate relation

* Normalization:-
- Normalization is a method to breaking down large complex table into smaller relates table and define relationship between them.

- Its reduce the redundancy of data (repeating data) in a table also remove inconsistency problem and unwanted risks.

* Advantages:-
- Maintain data integrity.
- Reduce the structure of table
- Avoid null values.
- Structuring of data so that model is flexible and easy to use.

* Types of Normalization:-
(i)- Normal form (1NF)
(ii)- Second Normal form (2NF)
(iii)- Third Normal form (3NF)
(iv)- Boyce Codd Normal form (BCNF)
(v)- Fourth Normal form (4NF)
(vi)- fifth Normal form (5NF)



(i)- First Normal form (1NF):-
- Data must be store in a table with rows and column and each column should contain individual values.

All values must be atomic (individual)

Ex- Project table

Sroll	Sname	Course	Teacher
1	Ankit	C, C++	Mr. A, Mr. B
2	Ankush	java, C	Mr. C, Mr. A
3	Neha	C++, java	Mr. B, Mr. C.

After 1NF

Sroll	Sname	Course	Teacher
1	Ankit	C	Mr. A
2	Ankush	C++	Mr. B
3	Neha		

After 1NF →

Sroll	Sname	Course	Teacher
1	Ankit	C	Mr. A
1	Ankit	C++	Mr. B
2	Ankush	java	Mr. C
2	Ankush	C	Mr. A
3	Neha	C++	Mr. B
3	Neha	java	Mr. C

(ii)- Second Normal form (2NF):-

- Table or relation must be in 1NF.
- All Non-key attributes should be fully dependend on the primary key (NO partial dependencies).

example- Project - table

Sroll	Sname	Course	Teacher
1	Ankit	C, C++	Mr. A, Mr. B
2	Ankush	java, C	Mr. C, Mr. A
3	Neha	C++, java	Mr. B, Mr. C.

After 1NF

Sroll	Sname	Course	Teacher
1	Ankit	C	Mr A
1	Ankit	C++	Mr B
2	Ankush	java	Mr. C
2	Ankush	C	Mr A
3	Neha	C++	Mr. B
3	Neha	java	Mr. C

13-10-25

After 2NF

Table 1 (Student)

Sroll	Sname
1	Ankit
2	Ankush
3	Neha

(Primary key)

Table 2 (Enrollment) (foreign key)

Sroll	Course	Teacher
1	C	Mr. A
1	C++	Mr. B
2	java	Mr. C
2	C	Mr. A
3	C++	Mr. B
3	java	Mr. C.

* Dr. Edger Frank Ted todd → Father of RDBMS

Imp → Codd's Rule

- The rules are called Codd's rule because they were introduced by Dr. Edger frank ted todd.

- He was a british Computer Scientist who is considered the father of RDBMS.

- Below is the summary of Codd's 12 rule:-

(i)- Codd's 0th rule:-
All the System must supports relational database feature It is also known as System rule

(ii)- Codd's 1st rule:-
According to this rule all the data must be stored in tabular format. It is also called information rule

(iii)- Codd's 2nd rule- In this rule a table must contain a column which can pick a record uniquely it means it is also known as Guaranteed access rule.

(iv)- Codd's 3rd rule → The system must handle null values. Null is a special type of value in terms of DBMS and it can be assign with any column of any datatype It is also known as NULL handling / Systematic treatment Value.

(v)- Codd's 4th rule → The database schema must be stored as relational model (table). It is also known as dynamic catalog / Descriptive data rule.

(vi)- Codd's 5th rule- Database must supports all the sublanguages of SQL (DDL, DML, DCL, DQL, TCL) It is also known as Comprehensive data sublanguage rule

(vii)- Codd's 6th rule - All the views that are theoratically a update must be updated by system. It is also known as View updatability rule.

(viii)- Codd's 7th rule- database must support SQL. insert, update, delete operation
It is also known as Set operation rule.

(ix)- Codd's 8th rule- Physical data changes should not affect how data is accessed
It is also known as Physical data independency rule.

(x)- Codd's 9th rule- Changes to the logical structure should not affect data access. It is also known as logical data independency rule

(xi)- Codd's 10th rule- Integrity constraints should be defines an schema not in application Code. It is also known as integrity dependency rule

(xii)- Codd's 11th rule → ~~Distribution~~ Distribution of data across location should be invisible to users It is also known as Distribution dependency rule

(xiii)- Codd's 12th rule → No By-passing integrity rule through low level access It is also known as Non Subversion rule

16-10-25

* Decomposition - Decomposition of relation is nothing but process of breaking a large relation (table) into two or more smaller relations to eliminated redundancy, alomalies and ensure, normalization

ex - Student (table)

Sid	Sname	Course	instructor
100	Ankit	C	Ravi
101	Ram	C++	Gopal
102	Ankit	Java	Jay
104	Ankush	C++	Neha

After Decomposed

student_info

Sid	Sname
100	Ankit
101	Ram
102	Ankit
104	Ankush

Course_info

Sid	Course	instructor
100	C	Ravi
101	C++	Gopal
102	Java	Jay
104	C++	Neha

* Types of Decomposition.

(i)- lossless Decomposition - It ensures, no data will be lost from original table during decomposition

ex- R table

Ename	ESkill	E_mail
Ankit	MERN	abc@gmail.com
Ashish	UI/UX	xyz@gmail.com
Rohit	PHP	123@gmail.com
Ankush	DBA	100@gmail.com

R1 table

Ename	ESkill
Ankit	MERN
Ashish	UI/UX
Rohit	PHP
Ankush	DBA

R2 table

Ename	E_mail
Ankit	abc@gmail.com
Ashish	xyz@gmail.com
Rohit	123@gmail.com
Ankush	100@gmail.com